3 Approaches

	Time	Space
• Brute force (nested loops)	$O(n^2)$	$O(1)$
• HashMap	$O(n)$	$O(n)$

↳ Use when need indices + unsorted array

• Two pointer	$O(n \log n)$	$O(1)$

↳ Use when need number or array already sorted.

- Lesson from HashMap approach
  - ↳ Earlier I used h.put(i, arr[i])
  - ↳ Which was wrong because .containsKey() return value associated with it and we could use indices & value to get to output
  - correct approach → h.put(arr[i], i)
- Lesson from Two pointer approach

```
while i<j:
    sum = a[i] + a[j]
    if sum == target → return answer
    if sum < target → i++ (need bigger sum)
    if sum > target → j-- (need smaller sum)

return new int[] {i,j}
```

Critical Rules:
- ↳ i<j (strict), not <= (both on same position means one value, not pair)
- ↳ Every iter^ must move one ptr
- ↳ Sorting alone cost $O(n \log n)$
- ↳ Prefer using time over space [space ↓ (minimise)]

Two pointer

↳ In this we use two indices (i, j) moving in coordinated directions on a sorted array / linked list to reduce time from $O(n^2)$ to $O(n)$ or $O(n \log n)$, with $O(1)$ space.

When to apply :

1) Array or Linked List question (not tree/graph / stack / dp)
2) Data is sorted or sorting would help.
3) Keywords : merge, remove duplicates, rearrange
4) Detect cycle in linked list
5) Find pairs / triplets / quadruplets (not a single element)
6) Constraint : no extra space allowed

Two pointer vs sliding window

↳ Different patterns. Similar looking but apply to different question types

- Sorting trap
  - ↳ Return Numbers : can sort freely (values don't change)
  - ↳ Return Indices : do NOT sort (sorting scrambles original indices), Use Hashmap instead, or works if array is sorted.